Experiment 2

Distributed Computing

Prof. Dhanashree Tamhane

Title: Develop RPC APIs to Enable Communication Between Services for your mini project

Aim - To develop and implement RPC APIs using gRPC for communication between distributed client and server services

Suppose my project is online shopping and build on Flask. I will first open my Git Bash (It follows Linux).

Step 1:- Go in your project folder. cd is the command to enter the folder.

Check your location by pwd command.

```
systems@VIT-LAPTOP-008 MINGW64 ~
$ cd flask-app
```

Step 2:- Check python version

Step 3:- Create a virtual environment

```
systems@VIT-LAPTOP-008 MINGW64 ~/flask-app
$ python -m venv venv
```

Step 4:- Activate the virtual environment. (venv shows its active)

```
systems@VIT-LAPTOP-008 MINGW64 ~/flask-app
$ source venv/Scripts/activate
(venv)
```

Step 5:- Install required packages

pip install grpcio grpcio-tools

Step 6:- Check pip install and install new version if needed.

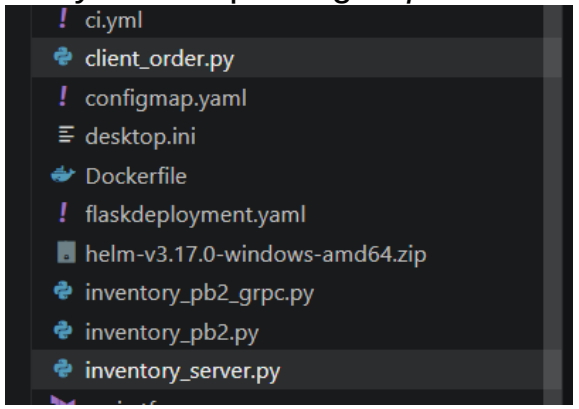
Step 7:- Verify pip install.

```
systems@VIT-LAPTOP-008 MINGW64 ~/flask-app
$ pip list
Package           Version
----------------- -------
grpcio            1.82.1
grpcio-tools      1.82.1
pip               26.1.2
protobuf          7.35.1
setuptools        83.0.0
typing_extensions 4.16.0
(venv)
```

Step 8:- Create file structure with the necessary grpc files.

```
|
├── proto/
|     inventory.proto
|
├── inventory_server.py
|
├── order_client.py
|
└── venv/
```

Step 9:- Create *.proto* file. Tell AI to give you a proto file based on your application. Add code for server.py and client.py. Also make sure that you are updating *requirements.txt* file. This is how my repo looks like-



I just created client_order.py and inventory_server.py. Added my code.

Make sure you are adding these 2 files are per your mini project application.

This is my requirements.txt for my Flask application.

requirements.txt

```
Flask==3.0.3
gunicorn==22.0.0
grpcio
grpcio-tools
protobuf
```

Step 10:- Generate python files. python -m grpc_tools.protoc -I=proto --python_out=. --grpc_python_out=. proto/inventory.proto

This generates

```
inventory_pb2.py
inventory_pb2_grpc.py
```

Step 11:- Now run the server.

python inventory_server.py

Output:

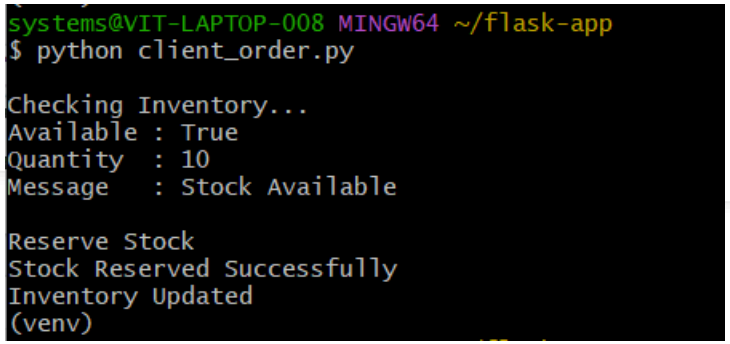
```
systems@VIT-LAPTOP-008 MINGW64 ~/flask-app
$ python inventory_server.py
Inventory Service Started on Port 50051...
```

Step 12:- Open new Git Bash window **(Do not close the server running bash window).** Go in your project window and just activate the virtual environment.

```
systems@VIT-LAPTOP-008 MINGW64 ~
$ cd flask-app
```

```
systems@VIT-LAPTOP-008 MINGW64 ~/flask-app
$ source venv/Scripts/activate
(venv)
```

Step 13:- Run the other service to make sure your connection is set up.



See the previous file. Those content are now running on my client server. Meaning – for my flask app, the content on Inventory_server.py are now transferred to my client_order server.

Conclusion (for my flask_app)**:**

The successful execution of the program confirmed that the Order Service was able to communicate with the Inventory Service using gRPC-based RPC calls. The client sent a request to check product availability, and the server processed the request and returned the inventory status. Since sufficient stock was available, the inventory was successfully reserved and updated.

The output verified that the RPC methods CheckStock(), ReserveStock(), and UpdateStock() were invoked correctly, demonstrating successful request-response communication between two distributed services.

Experiment 3

Distributed Computing

Prof. Dhanashree Tamhane

Title: Implementation of Lamport Logical Clock Synchronization to Maintain Consistent Event Timestamps in a Distributed System

This is for your python project. Javascript projects will follow the same flow, but they will have js supported files.

Step 1:- Go in your project folder. cd is the command to enter the folder.

Check your location by pwd command.

```
systems@VIT-LAPTOP-008 MINGW64 ~
$ cd flask-app
```

Step 2:- Create a new file called lamport_clock.py in your codebase for your mini project. Chatgpt will give you the script.

lamport_clock.py > LamportClock > receive_event

```python
class LamportClock:
    def __init__(self):
        self.time = 0

    def increment(self):
        self.time += 1
        return self.time

    def send_event(self):
        self.time += 1
        return self.time

    def receive_event(self, received):
        self.time = max(self.time, received) + 1
        return self.time
```

Step 3:- Update your client and server files with Lamport codes. Also update the proto file. There will be no change in the *requirements.txt* file. Make sure your files are for your mini project. Save all the files.

Step 4:- Run grpc support files again so Lamport gets initiated.
python -m grpc_tools.protoc -I=proto --python_out=. --grpc_python_out=. proto/inventory.proto

Step 5:- Create a virtual environment

```
systems@VIT-LAPTOP-008 MINGW64 ~/flask-app
$ python -m venv venv
```

Step 6:- Activate the virtual environment. (venv shows its active)

```
systems@VIT-LAPTOP-008 MINGW64 ~/flask-app
$ source venv/Scripts/activate
(venv)
```

Step 7:- Run the inventory server file.

```
systems@VIT-LAPTOP-008 MINGW64 ~/flask-app
$ python lamport_inventory_server.py
Inventory Service Started on Port 50051...

Received Request
Product ID : P101
Quantity   : 2
```

Step 8:- Open new Git Bash window **(Do not close the server running bash window).** Go in your project window and just activate the virtual environment.

Step 9:- Now run the client file.

```
systems@VIT-LAPTOP-008 MINGW64 ~/flask-app
$ python lamport_client_order.py

========== CHECK STOCK ==========
Sending Request
Lamport Clock : 1

Received Response
Lamport Clock : 2
Available      : True
Stock          : 10
Message        : Stock Available

========== RESERVE STOCK ==========
Sending Request
Lamport Clock : 3

Received Response
Lamport Clock : 4
Success        : True
Message        : Stock Reserved Successfully

========== UPDATE STOCK ==========
Sending Request
Lamport Clock : 5

Received Response
Lamport Clock : 6
Success        : True
Message        : Inventory Updated
(venv)
```

Conclusion :

The experiment successfully implemented Lamport Logical Clock Synchronization in a Flask-based e-commerce application using gRPC for communication between the Order Service and the Inventory Service. Lamport timestamps were attached to every RPC request and response, enabling both the client and server to maintain a consistent logical ordering of distributed events.

Upon receiving a message, each service updated its logical clock using the Lamport algorithm, ensuring that causally related events were processed in the correct sequence without relying on synchronized physical clocks.

The successful execution of the application demonstrated reliable communication between distributed services while preserving event ordering, illustrating the practical application of Lamport clocks in distributed systems for maintaining consistency, traceability, and coordination across multiple services.